

Orientation Guide to Predictive Action Projection with PPO

Safety–Performance Tradeoffs and Filter Dependence in Constrained Continuous Control

Purpose

This document gives a technically correct mental model of Project 1 before the reader enters the repository. It explains the problem, motivation, action-projection mechanism, PPO interaction, research questions, experimental logic, interpretation rules, major design decisions, claim boundaries, and recommended code-reading path. It is intentionally shorter than the implementation design and does not replace it.

Salvador Tenorio

Version 1.1 – September 2026

How to Use This Guide

The current implementation design is the authoritative implementation-facing description of Project 1, and the repository code remains the source of truth if prose and implementation disagree [1]. This guide is a shorter conceptual map: read it before entering the repository and reuse [section 10](#) when returning later.

Artifact	Purpose	Use it when
Orientation guide	Build the mental model, research rationale, and interpretation rules.	Beginning or refreshing.
Implementation design	Resolve exact mathematics, modules, data contracts, defaults, and change boundaries.	Inspecting or changing code.
Experiment records and separate paper	Audit what was run and communicate evidence, results, limitations, and conclusions.	Reviewing outcomes or writing.

Where the implementation design records an exact choice, this guide preserves it. Where it does not record every historical discussion, the guide gives the current methodological rationale rather than inventing a development transcript.

Reading target

One careful reading should take approximately 30–45 minutes. The five-minute overview in [section 1](#) and the refresher in [section 10](#) are deliberately self-contained.

Contents

How to Use This Guide	i
1 The Study in Five Minutes	1
2 The Navigation Task	2
3 Why an Action-Projection Layer Is Needed	3
4 Action Ownership and PPO Learning	4
5 Research Questions and Experimental Logic	5
6 Metrics and Interpretation	7
7 Why the Major Design Decisions Make Sense	8
8 What the Study Can and Cannot Establish	9
9 How to Enter the Repository	10
10 One-Page Refresher	10
A Symbols and Key Defaults	11

1 The Study in Five Minutes

1.1 The physical problem

A simulated mobile agent begins at a fixed start state and must reach a goal while avoiding circular obstacles. Its physical control is

$$u_t = \begin{bmatrix} v_t \\ \omega_t \end{bmatrix}, \quad 0 \leq v_t \leq v_{\max}, \quad -\omega_{\max} \leq \omega_t \leq \omega_{\max}, \quad (1)$$

where v_t is forward speed and ω_t is turn rate. A PPO policy observes the agent, goal, and obstacle geometry and proposes a control command.

A reward penalty can encourage the policy to avoid collisions, but it cannot prevent an unsafe command from being executed. Project 1 therefore inserts a local predictive safety filter between the policy and the environment.

1.2 The central idea

The policy proposes an unbounded normalized Gaussian action. The wrapper clips it componentwise to $[-1, 1]^2$ and maps the bounded action to physical units. If projection is enabled, a small quadratic program modifies the physical action only as much as needed to satisfy softened local obstacle constraints. The environment then advances under the executed action.

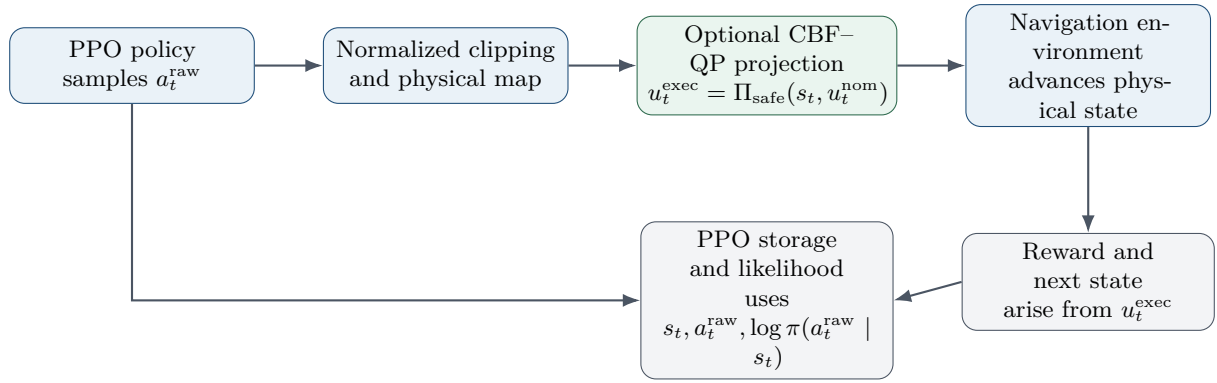


Figure 1: The central Project 1 action and learning path. The policy owns the sampled raw action; the environment advances under the executed physical action.

The single most important mental model

PPO owns the sampled normalized raw action and its log-probability. The wrapper stack converts it into a bounded physical nominal action and, when enabled, a projected physical executed action. Rewards and next states are consequences of the executed action. The distinction between what the policy proposed and what the environment executed is the central research object.

1.3 The research problem

Project 1 asks whether local predictive action projection can reduce measured collisions while preserving useful task behavior, and whether the learned policy itself becomes compatible with the filter or remains dependent on repeated correction.

The study is therefore not only about return. It examines three coupled quantities:

$$\boxed{\text{task performance} \quad \text{vs.} \quad \text{safety outcomes} \quad \text{vs.} \quad \text{projection burden}}. \quad (2)$$

1.4 The study in one paragraph

A CleanRL-style PPO agent is trained in a compact unicycle-navigation environment. Three implemented training variants provide a reference policy, a higher collision-penalty comparison, and a policy trained while the projection filter is active. Checkpoints can be evaluated with projection disabled or enabled, using 100 stochastic fixed-geometry episodes per checkpoint and mode for the primary evaluation and one deterministic actor-mean

execution on each of 24 prespecified layouts for transfer. The repository records complete episode outcomes, raw and executed actions, intervention frequency, correction magnitude, slack, solver status, trajectories, checkpoint and layout identities, and reproducible result artifacts. The local softened CBF-QP is treated as an empirical safety filter and measurement device, not as a global safety proof or global route planner [1].

2 The Navigation Task

2.1 State, observation, and obstacle encoding

The physical agent state is its planar position $p_t = (x_t, y_t)^\top$ and heading θ_t . The goal is $p_g \in \mathbb{R}^2$. Each obstacle has a center o_i , radius r_i , and active mask $m_i \in \{0, 1\}$. The agent has radius r_{agent} .

The policy receives a fixed-dimensional observation. With obstacle capacity N_{max} ,

$$d_{\text{obs}} = 6 + 5N_{\text{max}}. \quad (3)$$

The first six components encode

$$[x_t, y_t, \cos \theta_t, \sin \theta_t, x_g - x_t, y_g - y_t], \quad (4)$$

and each obstacle slot contributes relative position, signed collision clearance, physical radius, and active mask. Inactive slots are zero. The implemented default $N_{\text{max}} = 3$ produces a 21-dimensional observation and preserves checkpoint compatibility [1].

Why fixed obstacle capacity?

A neural network requires a stable input dimension. Separating obstacle *capacity* from the number of currently active obstacles permits controlled one-, two-, and three-obstacle layouts without changing the checkpoint architecture. Increasing the capacity is a different experiment because it changes the observation dimension and network compatibility.

2.2 Dynamics

The environment uses one explicit-Euler unicycle step with simulation interval Δt :

$$x_{t+1} = x_t + v_t \cos \theta_t \Delta t, \quad (5)$$

$$y_{t+1} = y_t + v_t \sin \theta_t \Delta t, \quad (6)$$

$$\theta_{t+1} = \text{wrap}(\theta_t + \omega_t \Delta t). \quad (7)$$

The model is deliberately compact: forward speed and turn rate create a genuine continuous-control problem while remaining simple enough to inspect analytically and in code.

2.3 Success, collision, and reward

The signed collision clearance from obstacle i is

$$d_{i,t}^{\text{clear}} = \|p_t - o_i\|_2 - (r_i + r_{\text{agent}}). \quad (8)$$

Positive clearance means separation, zero means boundary contact, and negative clearance means penetration. Collision occurs when the minimum active clearance is non-positive. Success occurs when the goal distance is within the goal radius.

Writing r_{t+1} for the reward emitted after executing $u_t^{\text{exec}} = [v_t, \omega_t]^\top$ from state t , the implemented reward is

$$\begin{aligned} r_{t+1} = & c_p(d_t^{\text{goal}} - d_{t+1}^{\text{goal}}) - c_u(v_t^2 + \omega_t^2) - c_{\text{time}} \\ & + r_g \mathbf{1}_{\text{success}, t+1} - c_c \mathbf{1}_{\text{collision}, t+1} - c_{\text{timeout}} d_{t+1}^{\text{goal}} \mathbf{1}_{\text{timeout}, t+1}. \end{aligned} \quad (9)$$

The terminal indicators refer to the post-transition state. The high-penalty training variant changes only c_c ; all final evaluations use the common collision penalty of 10.

Why this benchmark is appropriate

The environment is sufficiently structured to expose continuous-control, obstacle-avoidance, and policy-transfer behavior, but sufficiently small that every state feature, action transformation, reward term, and failure can be inspected. It is a research instrument, not a claim of realistic mobile-robot deployment.

3 Why an Action-Projection Layer Is Needed

3.1 Reward discouragement and execution-time intervention are different

A collision penalty changes the learning signal:

“Collisions should reduce expected return.”

It does not prevent the current policy from proposing and executing an unsafe action during training or evaluation.

Projection changes the executed control:

“This command will be replaced by the closest locally admissible command before the state advances.”

This distinction motivates the coexistence of the high-collision-penalty variant and the projection variants. They test different mechanisms: reward-based avoidance and direct action correction.

3.2 Why the projection is predictive

A center-only instantaneous check does not directly express how turn rate affects near-future motion. The projector therefore uses a point a distance L ahead of the agent:

$$p_L = p + L \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix}. \quad (10)$$

Its action Jacobian is

$$J_L(\theta) = \begin{bmatrix} \cos \theta & -L \sin \theta \\ \sin \theta & L \cos \theta \end{bmatrix}, \quad \dot{p}_L = J_L(\theta)u. \quad (11)$$

Both forward speed and turn rate now influence the motion of the monitored point. This is the sense in which the layer is *predictive*: it constrains the local motion of a point in front of the agent rather than merely detecting overlap after a step.

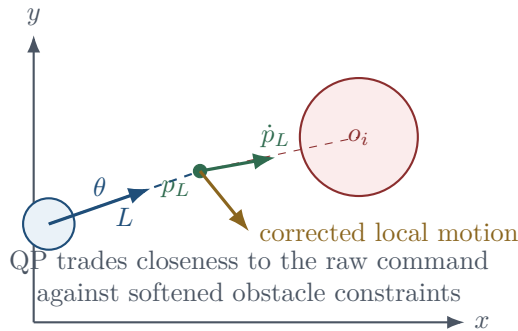


Figure 2: Conceptual lookahead geometry. The exact QP acts in control space; the arrows illustrate how the monitored point’s local motion can be redirected.

3.3 Barrier condition

For active obstacle i , define a projection radius

$$R_i^{\text{proj}} = r_i + r_{\text{agent}} + r_{\text{extra}} \quad (12)$$

and barrier function

$$h_i(p_L) = \|p_L - o_i\|_2^2 - (R_i^{\text{proj}})^2, \quad \nabla h_i = 2(p_L - o_i). \quad (13)$$

The implemented softened local condition is

$$\nabla h_i(p_L)^\top J_L(\theta)u + \xi_i \geq -\alpha h_i(p_L), \quad \xi_i \geq 0. \quad (14)$$

This construction follows the standard CBF–QP safety-filter pattern [2]. The inequality limits how rapidly the candidate action may reduce the barrier. Because $J_L(\theta)u$ is linear in $u = [v, \omega]^\top$, each obstacle contributes a linear constraint in the QP variables.

3.4 Projection QP

Given bounded physical nominal action u_t^{nom} , the executed action is obtained from

$$\begin{aligned} \min_{u, \xi} \quad & \frac{1}{2} \|u - u_t^{\text{nom}}\|_2^2 + \frac{\rho}{2} \|\xi\|_2^2 \\ \text{subject to} \quad & \text{the softened barrier rows in (14),} \\ & 0 \leq v \leq v_{\max}, \quad -\omega_{\max} \leq \omega \leq \omega_{\max}. \end{aligned} \quad (15)$$

The first term preserves the policy's intent by preferring the smallest correction. The second term penalizes violation of the local barrier inequalities through slack.

Why soft constraints and slack?

Hard constraints can make the QP infeasible when the state is already poor, several constraints conflict, or the local model cannot recover in one step. Slack makes the optimization numerically usable and turns the amount of violation into a measurable quantity. The price is equally important: a softened filter is not an absolute safety certificate. Large or persistent slack is evidence that the local safety mechanism is being asked to do more than it can guarantee.

3.5 Diagnostics and explicit failure behavior

The projection result records the bounded physical nominal action, physical executed action, correction vector and norm, intervention flag, slack vector and summaries, active-constraint count, solver status, success flag, and objective value. Persisted fields that retain “raw” in their names refer to this nominal physical action. These are scientific measurements, not incidental debugging fields.

If no active constraints exist, the projector returns the bounded raw action with success and zero slack. A successful QP returns finite action and slack values. A failed or invalid solve executes the stationary action $[0, 0]^\top$, reports failure, and leaves active slack unknown rather than pretending it was zero [1].

4 Action Ownership and PPO Learning

4.1 Four action representations

PPO samples an unbounded normalized action and the wrapper clips it:

$$a_t^{\text{raw}} \in \mathbb{R}^2, \quad a_t^{\text{bounded}} = \text{clip}(a_t^{\text{raw}}, -1, 1). \quad (16)$$

The bounded normalized action is mapped to the physical nominal command:

$$v_t^{\text{nom}} = \frac{1}{2} v_{\max} (a_{v,t}^{\text{bounded}} + 1), \quad (17)$$

$$\omega_t^{\text{nom}} = \omega_{\max} a_{\omega,t}^{\text{bounded}}. \quad (18)$$

Thus a normalized actor mean near $[0, 0]^\top$ maps to forward motion $[0.5v_{\max}, 0]^\top$, not a stationary command. The physical executed action is

$$u_t^{\text{exec}} = \begin{cases} u_t^{\text{nom}}, & \text{projection disabled,} \\ \Pi_{\text{safe}}(s_t, u_t^{\text{nom}}), & \text{projection enabled.} \end{cases} \quad (19)$$

Only u_t^{exec} reaches the base environment.

4.2 Wrapper order

The implemented construction order is

$$\begin{aligned} & \text{ConstrainedNavigationEnv} \\ & \rightarrow \text{CbfQpProjectionWrapper (optional)} \\ & \rightarrow \text{NormalizedActionWrapper (optional)} \\ & \rightarrow \text{RecordEpisodeStatistics (optional)}. \end{aligned} \quad (20)$$

Actions flow inward in reverse construction order. The policy emits normalized actions, normalization produces a bounded physical nominal action, and projection operates in physical action space before the base environment step.

4.3 What PPO stores

At rollout step t , PPO stores the policy sample and its likelihood data:

$$(s_t, a_t^{\text{raw}}, \log \pi_{\theta_{\text{old}}}(a_t^{\text{raw}} | s_t), r_{t+1}, d_t, V_t). \quad (21)$$

The PPO likelihood ratio is evaluated on the same sampled normalized raw action:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t^{\text{raw}} | s_t)}{\pi_{\theta_{\text{old}}}(a_t^{\text{raw}} | s_t)}. \quad (22)$$

Rewards and successor observations, however, are generated after a_t^{raw} has been mapped to u_t^{exec} . Generalized advantage estimation and the value targets therefore describe the projected closed loop whenever projection is active [6, 7].

Why the raw action is used in the likelihood

The policy sampled a_t^{raw} ; it did not sample u_t^{exec} . The executed action is the result of a deterministic, state-dependent transformation after sampling. Many raw actions may map to the same executed boundary action. Scoring the executed action as though it had been drawn directly from the original Gaussian would misidentify the policy random variable.

4.4 Effective projected environment

A compact way to understand the learning semantics is to define an effective environment whose action is still the policy's normalized raw action:

$$P_{\Pi}(s' | s, a) := P(s' | s, \Pi_{\text{safe}}(s, g(a))), \quad (23)$$

$$r_{\Pi}(s, a) := r(s, \Pi_{\text{safe}}(s, g(a))), \quad (24)$$

where g is the normalized-to-physical map. PPO is on-policy with respect to its nominal policy interacting with this effective safeguarded environment. Projection changes the closed-loop transition and reward map; it does not by itself create an unrelated behavior policy.

The implementation remains conservative. PPO's clipped surrogate is not presented as a proved unbiased projected-policy-gradient estimator. The correct description is an *empirical projected-environment formulation* with explicit raw/executed action ownership [3, 1].

4.5 Why PPO was an advantageous study instrument

PPO was not selected because SAC is invalid or because the code difference is inherently large. It was advantageous here because:

- fresh on-policy rollouts make the behavior distribution easy to identify;
- the sampled raw action appears explicitly in the likelihood ratio;
- the critic estimates state value rather than introducing learned raw-action gradients through a Q-function shaped by the many-to-one projection;
- the algorithm is inspectable in a CleanRL-style single-file training path;
- fewer interacting mechanisms make projection effects easier to attribute.

SAC could be a valid later robustness extension, particularly when sample efficiency is central. For P1, the priority was a transparent experimental instrument for studying the safety–performance–intervention relationship, not identifying the strongest general-purpose continuous-control learner [5, 4].

5 Research Questions and Experimental Logic

5.1 Primary question

Primary research question

Can a local predictive action-projection layer reduce measured safety violations in a constrained continuous-control benchmark while preserving useful task behavior, and what does that safety require from the filter?

5.2 Supporting questions

The primary question decomposes into the following operational questions:

1. What changes when projection is applied to an already-trained PPO checkpoint?
2. Does a policy trained while projection is active adapt to the safeguarded transition map?
3. Can a larger collision penalty reduce collisions without direct action intervention?
4. Does projection preserve success, or does it merely convert collisions into timeouts?
5. Does the policy learn locally admissible proposals, or does the filter repeatedly repair unsafe proposals?
6. Does the learned behavior transfer from the single training geometry to fixed alternative obstacle geometries?
7. When projection does not improve outcomes, do trajectories, corrections, slack, and solver diagnostics explain why?

5.3 Implemented training variants

The shared trainer implements three declared variants:

Method	Collision penalty	Projection during training	Scientific role
<code>ppo_baseline</code>	10	Disabled	Reference PPO learning under the ordinary environment action path.
<code>ppo_high_penalty</code>	50	Disabled	Reward-shaping comparison that changes only the collision cost.
<code>ppo_train_projection</code>	10	Enabled	Policy adaptation under transitions generated through the projection layer.

These are training conditions, not complete conclusions. The exact final evaluation groups and required projection modes are declared by the frozen analysis protocol rather than inferred from method names.

5.4 Evaluation-time projection and pairing

A final checkpoint can be evaluated with projection disabled and enabled under aligned seeds and environment settings. Same-final-checkpoint pairing holds the learned parameters fixed and measures the runtime-mode contrast without retraining the policy. It does not isolate a projector-only causal effect because the complete closed-loop trajectories can diverge. For stochastic evaluation, both modes begin each episode from the same environment seed and policy random state; trajectories may diverge after the first projected action changes the closed loop.

This design supports a clean distinction:

$$\underbrace{\text{how the policy was trained}}_{\text{learning condition}} \quad \text{and} \quad \underbrace{\text{whether projection is active during evaluation}}_{\text{deployment condition}}. \quad (25)$$

5.5 Deterministic and stochastic evaluation

The repository supports two PPO evaluation modes:

- **Deterministic actor mean:** the secondary 24-layout transfer evaluation, one execution per layout, checkpoint, and mode.
- **Stochastic policy sample:** the primary fixed-geometry evaluation, 100 complete episodes per checkpoint and mode.

Deterministic evaluation answers how the central tendency of the learned controller behaves. Stochastic evaluation reveals whether the policy distribution still places meaningful mass on actions requiring correction. Neither mode replaces the other.

5.6 Training layout and frozen core suite

Training uses one deterministic three-obstacle layout. A separate 24-layout core suite changes obstacle geometry while holding start, heading, goal, dynamics, reward, agent footprint, and episode limit fixed. It is a deterministic transfer and robustness benchmark, not evidence of broad task-distribution generalization. Its identity is computed from canonical parsed JSON so that line-ending changes do not alter the scientific layout identity [1].

Why freeze the layout suite?

A benchmark chosen after observing final results can accidentally favor a method or checkpoint. The core suite was calibrated as a policy-independent technical benchmark and then frozen. If a genuine defect is later found, it should receive a new version and all relevant methods should be rerun; favorable prior cases should not be selectively retained.

6 Metrics and Interpretation

6.1 Three families of evidence

Evidence family	Representative metrics	Question answered
Task behavior	Episode return, success, timeout, episode length, final goal distance	Does the controller accomplish the navigation task efficiently?
Safety behavior	Collision, minimum signed clearance, terminal safety outcome	Does the closed loop avoid or reduce geometric violations?
Filter burden	Intervention count/rate, mean and maximum correction, summed and maximum slack, solver failures	How often and by how much does runtime projection modify the nominal command?

6.2 Why return is insufficient

A single return number can mix several qualitatively different events. For example, OFF collision can correspond to ON timeout. Lower observed collision therefore need not accompany greater task completion. Conversely, a policy can obtain good return on the training geometry while failing under small obstacle changes.

The study reports matched outcomes from separate OFF/ON executions. These are descriptive correspondences, not physical conversions or isolated causal mechanisms:

Projection off	Projection on	Interpretation
Collision	Success	OFF collision corresponds to ON completion.
Collision	Timeout	OFF collision corresponds to ON noncompletion at the horizon.
Collision	Collision	Both executions collide; the outcomes alone do not identify delay or severity.
Success	Success	Both executions complete; burden metrics describe runtime correction.
Success	Timeout or collision	ON loses completion for this matched execution key.
Timeout	Success	OFF noncompletion corresponds to ON completion.
Timeout	Timeout	Neither execution completes within the horizon.

6.3 Filter dependence

Low collision rate does not prove that the policy learned safer nominal actions. A policy may repeatedly propose commands that the filter repairs. High intervention frequency, large correction magnitude, or persistent slack therefore indicates filter dependence.

A projection-trained policy should be inspected both with the filter active and, where required by the frozen protocol, with it removed. Degradation without the filter is not automatically a defect: a policy trained in the safeguarded environment may legitimately specialize to that closed loop. The scientific requirement is to measure and report that dependence rather than conceal it.

6.4 Statistical unit and aggregation

The independently initialized training run is the statistical replication unit, and one retained final checkpoint represents each run. Layouts are repeated test cases for that final checkpoint, not independent training replicates. The implemented aggregation therefore follows

$$\bar{y}_{m,j} = \frac{1}{L} \sum_{\ell=1}^L y_{m,j,\ell}, \quad \bar{y}_m = \frac{1}{J} \sum_{j=1}^J \bar{y}_{m,j}, \quad (26)$$

where layouts are averaged within each run’s retained final checkpoint before the five run-level quantities are weighted equally. This prevents layout-level pseudoreplication.

The completed study uses training seeds 1, 2, 3, 4, 5 for each of the three training histories. With five independent runs per history, individual run values, the equal-run mean, run-level sample standard deviation, range, sign consistency, and leave-one-run-out means are more defensible than strong universal or asymptotic claims.

7 Why the Major Design Decisions Make Sense

Interpretation

The table explains the present methodological logic of the implemented choices. It does not claim that every rationale was documented in this exact wording when the choice was first made.

Decision	Why this choice fits P1	Consequence or limitation
Compact custom benchmark	A single investigator can inspect every feature, reward term, collision, action transform, and trajectory; a large simulator would add realism before the core mechanism was understood.	Strong traceability; deliberately limited realism.
Unicycle action $[v, \omega]^\top$	It creates genuine continuous steering geometry while remaining analytically and computationally small.	Turning motivates the lookahead formulation; no higher-order vehicle dynamics.
Fixed obstacle capacity $N_{\max} = 3$	A stable neural input dimension avoids making set/attention architecture another research question.	At most three obstacle slots; changing capacity breaks checkpoint compatibility.
Normalized policy interface	PPO sees a symmetric action box while dynamics and projection remain in physical units.	Raw normalized, bounded physical nominal, and physical executed actions must be distinguished; normalized zero implies positive forward speed.
PPO rather than SAC	Fresh on-policy rollouts, explicit raw-action likelihoods, and fewer learned mechanisms make projection effects easier to attribute.	Usually less sample efficient than SAC; not a claim that PPO is universally better.
CleanRL-style implementation	The rollout, GAE, PPO loss, projection hook, logging, and checkpoint path remain visible in one inspectable algorithm.	Less framework generality, greater equation-to-code transparency.
Projection in physical space	Barrier rows, dynamics, and action limits share the same interpretable units.	The normalized-to-physical boundary becomes a strict ownership invariant.
CBF-QP rather than penalty alone	The QP directly changes unsafe commands, minimizes correction, and reports correction/slack burden.	It is a local safety filter, not a global planner or optimal safe-policy method.
Lookahead point	Both forward speed and turn rate affect monitored-point velocity.	Compact steering-aware constraint; behavior depends on lookahead and local geometry.
Soft constraints with slack	The QP remains usable when constraints conflict or recovery is locally impossible; slack quantifies relaxation.	No absolute safety certificate; large slack must be treated as evidence.
Stationary failure fallback	Solver failure is explicit and conservative rather than silently executing the raw command or reporting false zero slack.	Failures invalidate accepted final evidence and must be diagnosed.
Training-time and evaluation-time projection	Post-training safeguarding and adaptation under projected transitions are different scientific questions.	Training condition and deployment condition must be reported separately.
High collision-penalty baseline	Reward discouragement is the natural alternative to direct intervention.	Only the collision cost should differ in this comparison.
Deterministic and stochastic evaluation	Actor means give repeatable task/transfer trajectories; samples expose unsafe distribution tails.	Every result must identify the evaluation policy mode.

Decision	Why this choice fits P1	Consequence or limitation
One training layout plus frozen 24-layout suite	The study stays compact while still testing geometry transfer without outcome-driven case selection.	Transfer evidence is narrow; no broad task-distribution generalization.
Paired same-final-checkpoint evaluation	Aligned evaluation keys hold learned parameters fixed while the runtime mode changes.	Matched Projector ON minus Projector OFF outcomes are descriptive correspondences, not temporal transitions or projector-only causal effects.
Artifact-first analysis	Saved checkpoints, tables, trajectories, hashes, and audits separate evidence generation from presentation.	More up-front discipline; reproducible figures and rejection of stale/incomplete artifacts.
Run-level aggregation	Independently initialized training runs, not layouts or episodes, are the statistical replicates; one final checkpoint represents each run.	Avoids pseudoreplication; claims remain bounded with five runs per training history.
Research-code scope	Independent educational research prioritizes understanding, reproducibility, and mathematical correspondence over deployment architecture.	No ROS, hardware, service, or production-readiness claims.

8 What the Study Can and Cannot Establish

8.1 Claims that the evidence may support

Subject to the final results, Project 1 may support statements such as:

- projection changes measured collision, success, timeout, clearance, or return in the tested benchmark;
- the same checkpoint behaves differently with projection disabled and enabled;
- projection during training changes task behavior or intervention burden relative to ordinary training;
- a larger collision penalty produces a different safety–performance tradeoff from direct projection;
- the frozen layout suite reveals specialization, directional bias, transfer failure, or projection compensation;
- intervention, correction, slack, and trajectories identify when the filter rather than the policy carries the safety behavior.

8.2 Claims that are outside the design

Project 1 does not establish:

- global forward invariance for arbitrary states and policies;
- a global collision-free route planner;
- formal optimality of the projected PPO policy;
- a proved unbiased clipped PPO estimator under action projection;
- real-time, hardware, robotics-middleware, or deployment readiness;
- superiority over all safe-RL, constrained-RL, MPC, or continuous-control algorithms;
- broad generalization beyond the compact simulator and declared layouts.

A crucial interpretation rule

“Projection prevented a collision” and “the policy learned safe behavior” are different claims. The first concerns the safeguarded closed loop. The second requires evidence that nominal proposals themselves need less correction. Project 1 records both outcomes precisely so that they are not conflated.

8.3 Expected failure modes are informative

Several negative outcomes remain scientifically useful:

- **Collision becomes timeout:** local safety improves, but route completion does not.
- **Persistent intervention:** the policy relies on the filter.
- **Large slack:** the local condition is being relaxed substantially.

- **Solver failure:** the numerical safety mechanism is unreliable under that configuration.
- **Training-layout success but suite failure:** the policy specialized to the training geometry.
- **Projection harms an otherwise successful route:** the filter is locally conservative or poorly aligned with task progress.

The research objective is not to guarantee that projection “wins” every comparison. It is to characterize the mechanisms and tradeoffs accurately.

9 How to Enter the Repository

Do not browse alphabetically. Follow physical ownership and data flow:

Step	Code anchor	What to understand before continuing
1	<code>environments/constrained_navigation.py</code>	Observation, unicycle dynamics, reward, success, collision, truncation, built-in layout, and controlled reset layouts.
2	<code>environments/action_wrappers.py</code>	Normalized-to-physical mapping and why normalized $[0, 0]^\top$ means forward motion.
3	<code>environments/factory.py</code>	Exact wrapper nesting and which layer owns each action representation.
4	<code>projection/cbf_qp_projection.py</code>	Lookahead geometry, barrier rows, QP objective, bounds, slack, special paths, and failure fallback.
5	<code>projection/cbf_qp_wrapper.py</code>	How current geometry is read, u^{exec} reaches the environment, and diagnostics enter <code>info</code> .
6	<code>algorithms/ppo/agent.py</code>	Gaussian actor mean/log standard deviation and the state-value critic.
7	<code>algorithms/ppo/ppo_continuous_action.py</code>	One complete rollout and update: sample a^{raw} , call <code>envs.step</code> , store likelihood data, compute GAE, update PPO.
8	<code>experiments/train_ppo_variant.py</code>	Exactly which settings differ among baseline, high-penalty, and projection-trained methods.
9	<code>evaluation/evaluate_projection_pair.py</code>	Same-checkpoint, aligned-seed projection-off/on evaluation and paired artifacts.
10	<code>evaluation/evaluate_layout_suite.py</code> ; <code>evaluation/layout_suite.py</code>	Frozen suite validation, semantic hashing, layout identity, and per-layout evidence.
11	<code>evaluation/trajectory_recording.py</code>	$T + 1$ state / T transition alignment among states, raw and executed actions, rewards, and diagnostics.
12	<code>analysis/build_projection_results.py</code> and supporting analysis modules	Completeness audits, run-level final-checkpoint aggregation, saved-artifact-only figures, and generated tables.

Working method

At every step ask: What does this module own? Which action representation does it receive and return? What research evidence does it generate? Which invariant would be violated by an incorrect change?

10 One-Page Refresher

P1 in one sentence

A PPO policy proposes continuous navigation actions, a local softened CBF-QP optionally replaces unsafe physical commands with nearby alternatives, and the study measures whether collision reduction is obtained without unacceptable loss of task performance or excessive dependence on the filter.

Central action chain

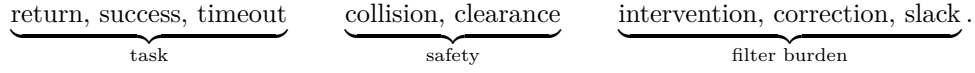
$$a_t^{\text{raw}} \xrightarrow{\text{clip}} a_t^{\text{bounded}} \xrightarrow{\text{physical map}} u_t^{\text{nom}} \xrightarrow{\Pi_{\text{safe}}(s_t, \cdot)} u_t^{\text{exec}} \xrightarrow{\text{environment}} (r_{t+1}, s_{t+1}).$$

PPO scores a_t^{raw} . The environment advances under u_t^{exec} .

Five questions to remember

1. Does projection reduce collisions for the same checkpoint?
2. Does projection preserve completion, or coincide mainly with more timeout?
3. Does training with projection reduce later correction burden?
4. How does reward-based collision discouragement compare with direct action correction?
5. Does behavior transfer to changed obstacle geometry, and how dependent is it on the filter?

Three evidence families



Three claim boundaries

- Local softened projection is not a global safety guarantee.
- Projected PPO is an empirical projected-environment formulation, not a claimed unbiased corrected estimator.
- The fixed training layout and 24-layout suite do not establish broad real-world generalization.

First files to reopen

Task and dynamics	<code>environments/constrained_navigation.py</code>
Action ownership	<code>environments/action_wrappers.py</code> ; <code>projection/cbf_qp_wrapper.py</code>
Projection QP	<code>projection/cbf_qp_projection.py</code>
PPO learning path	<code>algorithms/ppo/ppo_continuous_action.py</code>
Evidence and results	<code>evaluation/evaluate_projection_pair.py</code> ; <code>analysis/build_projection_results.py</code>

A Symbols and Key Defaults

Symbol	Meaning	Key default	Value
s_t	Policy observation.	Obstacle capacity	3
p_t, θ_t	Agent position and heading.	Active training obstacles	3
a_t^{raw}	Normalized action sampled by PPO.	Δt	0.1
u_t^{nom}	Bounded physical nominal action before projection.	$v_{\text{max}}, \omega_{\text{max}}$	1.0, 2.0
u_t^{exec}	Physical action executed by the environment.	Episode limit	200
Π_{safe}	State-dependent physical projection map.	Collision penalties	10, 50
p_L, J_L	Lookahead point and action Jacobian.	Lookahead L	0.25
h_i, ξ_i	Obstacle barrier and nonnegative slack.	α, ρ	2, 1000
$\ u^{\text{exec}} - u^{\text{raw}}\ _2$	Projection correction magnitude.	Extra clearance	0.0
P_Π, r_Π	Effective transition and reward maps.	Training seeds	1, 2, 3, 4, 5

The implementation design and saved run metadata remain authoritative for exact values. The implementation design records a core budget of 51,200 transitions, goal radius 0.25, agent radius 0.10, and intervention tolerance 10^{-6} [1].

References

- [1] Salvador Tenorio. *Constrained Policy Learning with Predictive Action Projection: Implementation Design*. Technical design document, 2026.
- [2] Aaron D. Ames, Xiangru Xu, Jessy W. Grizzle, and Paulo Tabuada. Control barrier function based quadratic programs for safety critical systems. *IEEE Transactions on Automatic Control*, 62(8):3861–3876, 2017. doi: [10.1109/TAC.2016.2638961](https://doi.org/10.1109/TAC.2016.2638961).
- [3] Sébastien Gros, Mario Zanon, and Alberto Bemporad. Safe reinforcement learning via projection on a safe set: How to achieve optimality? *IFAC-PapersOnLine*, 53(2):8076–8081, 2020. doi: [10.1016/j.ifacol.2020.12.2276](https://doi.org/10.1016/j.ifacol.2020.12.2276).
- [4] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine. Soft Actor-Critic algorithms and applications. arXiv:1812.05905, 2018.
- [5] Shengyi Huang, Rousslan Fernand Julien Dossa, Chang Ye, Jeff Braga, Dipam Chakraborty, Kinal Mehta, and João G. M. Araújo. CleanRL: High-quality single-file implementations of deep reinforcement learning algorithms. *Journal of Machine Learning Research*, 23(274):1–18, 2022.
- [6] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.
- [7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017.